

Sistema Colaborativo de Comparação de Preços por Cupons Fiscais

Instituição: Faculdade de Nova Serrana — FANS

Curso: Engenharia de Software — 5º Período

Contexto: Projeto Prático — Avaliação entre turmas

Versão: 1.0

Sumário

1. Visão Geral do Projeto
 2. Objetivos
 3. Stack Tecnológica
 4. Arquitetura do Sistema
 5. Modelagem de Dados
 6. API REST — Endpoints
 7. Funcionalidades Obrigatórias
 8. Fluxo Principal da Aplicação
 9. Fases de Desenvolvimento
 10. Estrutura de Diretórios
 11. Configuração e Execução
-

1. Visão Geral do Projeto

O sistema permite que usuários escaneiem o QR Code de cupons fiscais eletrônicos (NFC-e) e, a partir disso, tenham acesso a um histórico colaborativo de preços praticados por diferentes estabelecimentos. A proposta é transformar dados de compras cotidianas em inteligência de consumo compartilhada.

A aplicação opera em duas fases distintas: primeiramente, a construção da API back-end responsável por processar, armazenar e servir os dados; e, em seguida, o consumo dessa API por uma interface front-end desenvolvida em React.

2. Objetivos

2.1 Objetivo Principal

Desenvolver uma aplicação full-stack capaz de capturar, processar e compartilhar informações de preços extraídas de cupons fiscais eletrônicos (NFC-e), permitindo ao usuário consultar e comparar preços de produtos em diferentes estabelecimentos ao longo do tempo.

2.2 Objetivos Específicos

- Implementar leitura de QR Code e captura do HTML da NFC-e
- Construir uma API REST com Node.js + Express consumindo e servindo dados em JSON
- Persistir dados em MongoDB com schemas definidos via Mongoose
- Criar interface web em React que consome a API
- Implementar as quatro operações CRUD completas
- Oferecer consultas de histórico e comparação de preços por produto

3. Stack Tecnológica

3.1 Back-end

Tecnologia	Versão recomendada	Papel
Node.js	20.x LTS	Runtime JavaScript server-side
Express	4.x	Framework HTTP para a API REST
Mongoose	8.x	ODM — mapeamento objetos ↔ MongoDB
dotenv	16.x	Gerenciamento de variáveis de ambiente
cors	2.x	Controle de acesso cross-origin
bcryptjs	2.x	Hash de senhas dos usuários
jsonwebtoken	9.x	Autenticação via JWT
axios	1.x	Requisições HTTP para buscar HTML da NFC-e
cheerio	1.x	Parsing do HTML da NFC-e (server-side)

3.2 Banco de Dados

Tecnologia	Papel
MongoDB	Banco NoSQL orientado a documentos (obrigatório)
MongoDB Atlas	Hospedagem em nuvem (recomendado para desenvolvimento)

Justificativa do MongoDB: O esquema de itens de compra é naturalmente variável — cada cupom pode ter quantidades distintas de produtos. O modelo de documentos do MongoDB

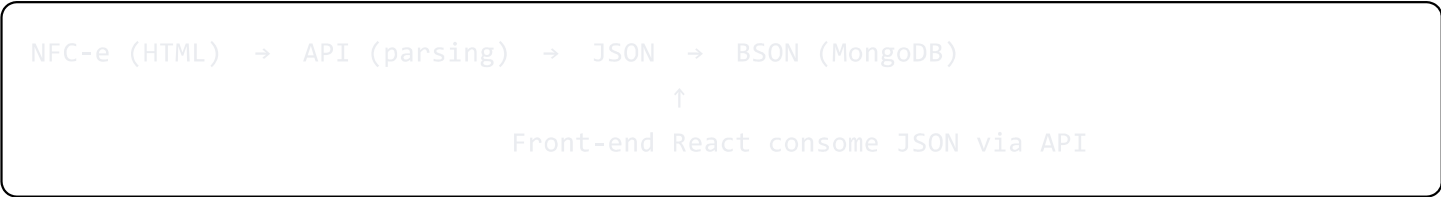
se adapta melhor a essa realidade do que um esquema relacional rígido. O formato BSON (Binary JSON) mantém fidelidade com o JSON servido pela API.

3.3 Front-end

Tecnologia	Papel
React 18	Biblioteca de UI (selecionada sobre Vue e Angular)
Vite	Build tool e dev server
Axios	Consumo da API REST
React Router	Navegação entre páginas

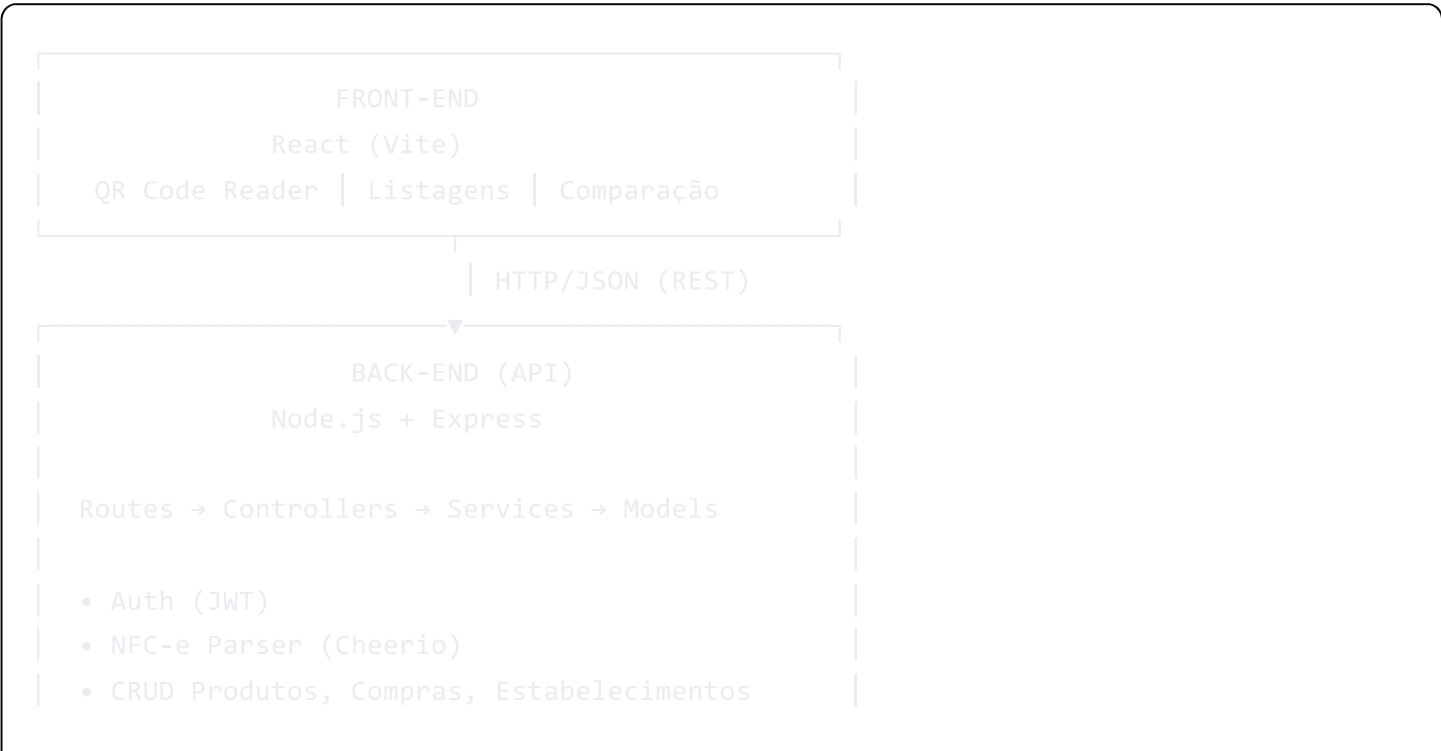
Justificativa do React: Maior facilidade de escalar para mobile via React Native sem reescrever a lógica de negócio. Ecossistema robusto e maior demanda no mercado. Curva de aprendizado comparável ao Vue para times com background JavaScript.

3.4 Fluxo de Dados



4. Arquitetura do Sistema

4.1 Visão em Camadas





4.2 Padrão Arquitetural

O back-end segue o padrão **MVC adaptado para APIs**:

- **Routes:** mapeiam URLs para controllers
- **Controllers:** recebem a requisição, delegam para services, retornam a resposta
- **Services:** contêm a lógica de negócio (parsing de NFC-e, deduplicação de produtos)
- **Models:** schemas Mongoose que interagem com o MongoDB

5. Modelagem de Dados

5.1 Schema: Usuário

javascript

```
const usuarioSchema = new mongoose.Schema({
  nome: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  senha: { type: String, required: true }, // armazenada com bcrypt
  criado_em: { type: Date, default: Date.now }
});
```

5.2 Schema: Estabelecimento

javascript

```
const estabelecimentoSchema = new mongoose.Schema({
  nome: { type: String, required: true, trim: true },
  cnpj: { type: String, required: true, unique: true },
  endereco: { type: String },
  criado_em: { type: Date, default: Date.now }
});
```

5.3 Schema: Produto

javascript

```
const produtoSchema = new mongoose.Schema({
  nome:      { type: String, required: true, trim: true },
  categoria:{ type: String },
  menor_preco: { type: Number, default: null },
  ultimo_preco: {
    valor:      { type: Number },
    data:       { type: Date },
    estabelecimento_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Estabelecimento' }
  },
  criado_em: { type: Date, default: Date.now }
});
```

5.4 Schema: Compra

javascript

```
const itemCompraSchema = new mongoose.Schema({
  produto_id:    { type: mongoose.Schema.Types.ObjectId, ref: 'Produto', required: true
  nome_original: { type: String },                               // nome como consta no cupom
  quantidade:    { type: Number, required: true },
  valor_unitario:{ type: Number, required: true },
  valor_total:   { type: Number, required: true }
}, { _id: false });

const compraSchema = new mongoose.Schema({
  usuario_id:      { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
  estabelecimento_id:{ type: mongoose.Schema.Types.ObjectId, ref: 'Estabelecimento', required: true },
  data_compra:     { type: Date, required: true },
  valor_total:     { type: Number, required: true },
  nfce_url:        { type: String },
  itens:           [itemCompraSchema],
  criado_em:       { type: Date, default: Date.now }
});
```

5.5 Schema: Histórico de Preços

javascript

```
const historicoPrecoSchema = new mongoose.Schema({
  produto_id:      { type: mongoose.Schema.Types.ObjectId, ref: 'Produto', required: true },
  estabelecimento_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Estabelecimento', required: true },
  compra_id:      { type: mongoose.Schema.Types.ObjectId, ref: 'Compra', required: true },
  valor:          { type: Number, required: true },
  data:           { type: Date, required: true }
});
```

6. API REST — Endpoints

6.1 Autenticação

Método	Rota	Descrição	Auth?
POST	<code>/api/auth/register</code>	Cadastrar novo usuário	Não
POST	<code>/api/auth/login</code>	Login — retorna JWT	Não

Exemplo — POST `/api/auth/login`

Request body:

```
json
{
  "email": "usuario@email.com",
  "senha": "minhasenha123"
}
```

Response `200 OK`:

```
json
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "usuario": {
    "id": "64a1b2c3d4e5f6789012345",
    "nome": "João Silva",
    "email": "usuario@email.com"
  }
}
```

6.2 Processamento de NFC-e

Método	Rota	Descrição	Auth?
POST	<code>/api/nfce/processar</code>	Envia HTML da NFC-e para extração	Sim

Exemplo — POST `/api/nfce/processar`

Request body:

```
json
{
  "html": "<html>... conteúdo da página NFC-e ...</html>",
  "url_origem": "https://www.sefaz.mg.gov.br/nfce/..."
}
```

Response `201 Created`:

```
json
{
  "compra_id": "64a1b2c3d4e5f6789012346",
  "estabelecimento": "SUPERMERCADO ABC LTDA",
  "data_compra": "2025-06-01T14:30:00Z",
  "valor_total": 87.45,
  "itens_processados": 12,
  "itens_novos": 3
}
```

6.3 Produtos

Método	Rota	Descrição	Auth?
GET	<code>/api/produtos</code>	Listar produtos (com filtro por nome)	Não
GET	<code>/api/produtos/:id</code>	Detalhes + histórico de preços de um produto	Não
POST	<code>/api/produtos</code>	Criar produto manualmente	Sim
PUT	<code>/api/produtos/:id</code>	Atualizar produto	Sim
DELETE	<code>/api/produtos/:id</code>	Remover produto	Sim

Exemplo — GET `/api/produtos?nome=arroz`

Response `200 OK`:

```
json
{
  "produtos": [
    {
      "id": "64a1b2c3d4e5f6789012347",
      "nome": "ARROZ TIOJOAO 5KG",
      "categoria": "Grãos",
      "menor_preco": 22.90,
      "ultimo_preco": {
        "valor": 24.50,
        "data": "2025-06-01T14:30:00Z",
        "estabelecimento": "SUPERMERCADO ABC"
      }
    }
  ]
}
```

Exemplo — GET /api/produtos/:id (histórico completo)

Response `200 OK`:

```
json
{
  "id": "64a1b2c3d4e5f6789012347",
  "nome": "ARROZ TIOJOAO 5KG",
  "menor_preco": 22.90,
  "ultimo_preco": 24.50,
  "historico": [
    { "valor": 24.50, "estabelecimento": "SUPERMERCADO ABC", "data": "2025-06-01" },
    { "valor": 23.80, "estabelecimento": "ATACADÃO XYZ", "data": "2025-05-20" },
    { "valor": 22.90, "estabelecimento": "ATACADÃO XYZ", "data": "2025-05-05" }
  ]
}
```

6.4 Compras

Método	Rota	Descrição	Auth?
GET	<code>/api/compras</code>	Listar compras do usuário autenticado	Sim
GET	<code>/api/compras/:id</code>	Detalhes de uma compra específica	Sim

Método	Rota	Descrição	Auth?
POST	<code>/api/compras</code>	Registrar compra manualmente	Sim
PUT	<code>/api/compras/:id</code>	Atualizar dados de uma compra	Sim
DELETE	<code>/api/compras/:id</code>	Remover compra	Sim

6.5 Estabelecimentos

Método	Rota	Descrição	Auth?
GET	<code>/api/estabelecimentos</code>	Listar estabelecimentos	Não
GET	<code>/api/estabelecimentos/:id</code>	Detalhes de um estabelecimento	Não
POST	<code>/api/estabelecimentos</code>	Criar estabelecimento	Sim
PUT	<code>/api/estabelecimentos/:id</code>	Atualizar estabelecimento	Sim
DELETE	<code>/api/estabelecimentos/:id</code>	Remover estabelecimento	Sim

6.6 Resumo CRUD × HTTP

Operação	Verbo HTTP	Exemplo de rota
Create	POST	<code>POST /api/produtos</code>
Read	GET	<code>GET /api/produtos/:id</code>
Update	PUT	<code>PUT /api/produtos/:id</code>
Delete	DELETE	<code>DELETE /api/produtos/:id</code>

7. Funcionalidades Obrigatórias

7.1 Leitura do QR Code

O front-end deve permitir que o usuário escaneie o QR Code presente no cupom fiscal. O QR Code contém a URL da consulta pública da NFC-e junto à SEFAZ do estado. O resultado do escaneamento é a URL que será acessada para obtenção do HTML.

Biblioteca sugerida (front-end React): `html5-qrcode` ou `@zxing/library`

7.2 Captura do HTML da NFC-e

Após obter a URL, o aplicativo faz uma requisição HTTP para a página pública da NFC-e e captura o HTML retornado. **Importante:** essa requisição deve ser feita pelo back-end (Node.js) para evitar bloqueios de CORS no navegador.

Fluxo:

1. Front-end envia a URL ao endpoint `POST /api/nfce/processar`
2. A API acessa a URL com `axios.get(url)`
3. O HTML retornado é processado com Cheerio

7.3 Processamento do Cupom (Parsing)

A API extrai do HTML da NFC-e:

Campo	Localização típica no HTML
Nome do estabelecimento	<code><div class="txtTopo"></code> ou <code></code> com razão social
CNPJ	Junto ao nome do estabelecimento
Data da emissão	Campo de data/hora na NFC-e
Valor total	Seção de totais da nota
Itens (nome, qtd, valor)	Tabela de produtos

Atenção: A estrutura do HTML pode variar por estado e sistema emissor. O parser deve ser resiliente a pequenas variações de markup.

7.4 Cadastro Automático de Produtos

Lógica ao processar cada item do cupom:

Para cada item do cupom:

1. Buscar no MongoDB: `Produto.findOne({ nome: { $regex: nome_item, $options: 'i' } })`
2. Se encontrado → usar o produto existente (associar compra)
3. Se não encontrado → criar novo produto: `Produto.create({ nome: nome_item })`
4. Registrar o preço no histórico
5. Atualizar `menor_preco` e `ultimo_preco` no documento do produto

7.5 Histórico de Compras

Endpoint `GET /api/compras` retorna as compras do usuário autenticado com:

- Lista de produtos comprados por compra
- Data de cada compra
- Valor pago por cada item
- Estabelecimento onde foi realizada a compra

7.6 Consulta e Comparação de Preços

Endpoint `GET /api/produtos/:id` retorna:

- Último preço registrado (e onde foi encontrado)
 - Menor preço registrado (e onde foi encontrado)
 - Histórico completo de preços com datas e estabelecimentos
-

8. Fluxo Principal da Aplicação



9. Fases de Desenvolvimento

Fase 1 — Back-end (API REST)

- ☐ Configuração do projeto Node.js (npm init, instalação de dependências)
- ☐ Conexão com MongoDB via Mongoose
- ☐ Definição dos schemas (Usuario, Produto, Compra, Estabelecimento, HistoricoPreco)
- ☐ Implementação das rotas de autenticação (register, login, JWT)
- ☐ Implementação do CRUD de Produtos
- ☐ Implementação do CRUD de Compras
- ☐ Implementação do CRUD de Estabelecimentos
- ☐ Implementação do endpoint de processamento de NFC-e
- ☐ Implementação do parser HTML com Cheerio
- ☐ Testes dos endpoints via Postman/Insomnia

Fase 2 — Front-end (Consumo da API)

- ☐ Setup do projeto React com Vite
- ☐ Configuração do Axios com baseURL e interceptors (JWT header)
- ☐ Telas: Login, Cadastro
- ☐ Tela: Escaneamento de QR Code
- ☐ Tela: Histórico de compras do usuário
- ☐ Tela: Busca de produtos + comparação de preços
- ☐ Tela: Detalhes do produto (histórico de preços)

10. Estrutura de Diretórios

Back-end

```
backend/  
├── src/  
│   ├── config/  
│   │   └── database.js          # Conexão MongoDB  
│   ├── controllers/  
│   │   ├── authController.js  
│   │   ├── produtoController.js  
│   │   ├── compraController.js  
│   │   └── estabelecimentoController.js
```

```
|   |   └─ nfceController.js
|   └─ middleware/
|       └─ authMiddleware.js      # Validação JWT
|   └─ models/
|       └─ Usuario.js
|       └─ Produto.js
|       └─ Compra.js
|       └─ Estabelecimento.js
|       └─ HistoricoPreco.js
|   └─ routes/
|       └─ authRoutes.js
|       └─ produtoRoutes.js
|       └─ compraRoutes.js
|       └─ estabelecimentoRoutes.js
|       └─ nfceRoutes.js
|   └─ services/
|       └─ nfceParser.js          # Lógica de parsing do HTML
|   └─ app.js                    # Express app + routes
└─ .env
└─ .env.example
└─ package.json
└─ server.js                    # Entry point
```

Front-end

```
frontend/
└─ src/
    └─ components/
        └─ QRCodeScanner.jsx
        └─ ProductCard.jsx
        └─ PriceHistory.jsx
    └─ pages/
        └─ Login.jsx
        └─ Dashboard.jsx
        └─ Scanner.jsx
        └─ Products.jsx
        └─ ProductDetail.jsx
    └─ services/
        └─ api.js                # Configuração Axios
    └─ context/
        └─ AuthContext.jsx      # Contexto de autenticação
    └─ App.jsx
└─ .env
└─ package.json
```

11. Configuração e Execução

11.1 Pré-requisitos

- Node.js 20.x LTS
- MongoDB (local ou MongoDB Atlas)
- npm ou yarn

11.2 Back-end

```
bash

# Clonar o repositório
git clone <url-do-repositorio>
cd backend

# Instalar dependências
npm install

# Configurar variáveis de ambiente
cp .env.example .env
# Editar .env com suas credenciais
```

Arquivo .env:

```
PORT=3001
MONGODB_URI=mongodb://localhost:27017/comparador_precos
JWT_SECRET=sua_chave_secreta_aqui
JWT_EXPIRES_IN=7d
```

```
bash

# Executar em desenvolvimento
npm run dev

# Executar em produção
npm start
```

11.3 Front-end

```
bash
```

```
cd frontend
npm install

# Configurar variável de ambiente
echo "VITE_API_URL=http://localhost:3001" > .env
```

Arquivo .env:

```
VITE_API_URL=http://localhost:3001/api
```

```
bash

# Executar em desenvolvimento
npm run dev

# Build para produção
npm run build
```

11.4 Dependências do Back-end (package.json)

```
json

{
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.0.0",
    "dotenv": "^16.3.1",
    "cors": "^2.8.5",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.2",
    "axios": "^1.6.0",
    "cheerio": "^1.0.0-rc.12"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  },
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

Considerações de Segurança

- Senhas armazenadas com hash bcrypt (salt rounds: 10)
 - Autenticação via JWT com expiração configurável
 - Middleware de autenticação aplicado a rotas que alteram dados
 - Validação de entrada nos controllers antes de persistir no banco
 - CORS configurado para aceitar apenas origens conhecidas em produção
-

Glossário

Termo	Definição
NFC-e	Nota Fiscal de Consumidor Eletrônica — documento fiscal digital emitido no varejo
QR Code	Código de barras 2D presente no cupom que contém a URL de consulta da NFC-e
BSON	Binary JSON — formato binário usado internamente pelo MongoDB
ODM	Object Document Mapper — equivalente ao ORM para bancos de documentos
JWT	JSON Web Token — padrão de autenticação stateless via token assinado
CRUD	Create, Read, Update, Delete — as quatro operações básicas de persistência
Parsing	Extração e estruturação de dados a partir de conteúdo não estruturado (HTML)

Documentação gerada para o projeto de avaliação prática — FANS, 5º Período de Engenharia de Software.